

Tarea Corta 4 – Listas Circulares

Esta tarea consiste en la implementación, utilización y modificación de las siguientes estructuras de datos:

1. Lista circular simplemente enlazada
2. Lista circular doblemente enlazada

Parte 1

Implementar la clase abstracta **CList**. Esta clase servirá de base para la implementación de las clases concretas más adelante. Deben seguirse los mismos lineamientos vistos en clases para los tipos abstractos: debe ser genérica, debe eliminar el constructor de copia y la operación de asignación, debe tener un constructor vacío, un destructor vacío marcado como virtual y el resto de los métodos que se detallan a continuación, todos marcados como virtual y sin implementación (=0):

1. **void insert(E element)**
Método que inserta un elemento en el frente de la lista. Si se invoca el método `getFront` después de esta inserción, debería dar como resultado el mismo elemento insertado.
2. **void insertBack(E element)**
Método que inserta un elemento en el final de la lista. Si se invoca el método `getBack` después de esta inserción, debería dar como resultado el mismo elemento insertado.
3. **E remove()**
Elimina el elemento que se encuentra en el frente de la lista. Retorna como resultado el elemento que se eliminó. Debe lanzar un error en caso de que la lista se encuentre vacía.
4. **E removeBack()**
Elimina el elemento que se encuentra en el final de la lista. Retorna como resultado el elemento que se eliminó. Debe lanzar un error en caso de que la lista se encuentre vacía.
5. **void clear()**
Elimina todos los contenidos de la lista y la deja vacía.
6. **E getFront()**
Retorna el elemento ubicado en el frente de la lista. Debe lanzar un error en caso de que la lista se encuentre vacía.
7. **E getBack()**
Retorna el elemento ubicado en el final de la lista. Debe lanzar un error en caso de que la lista se encuentre vacía.
8. **void next()**
Mueve la posición actual a la posición siguiente.
9. **void previous()**
Mueve la posición actual a la posición anterior.
10. **int getSize()**

Retorna un número entero con la cantidad de elementos que se encuentran en la lista actualmente.

11. void print()

Imprime en pantalla los contenidos de la lista, iniciando por el elemento en el frente de la lista y terminando con el elemento en el final de la lista.

Parte 2

Implementar las clases **CircleList** y **DCircleList**, ambas son listas circulares enlazadas, la primera es con enlaces simples y la segunda con dobles. Las clases deben heredar de la clase abstracta **CList** e implementar todos sus métodos. La clase **CircleList** debe utilizar la clase **Node** para guardar los elementos, mientras que la clase **DCircleList** debe utilizar la clase **DNode**. Ambas clases son las mismas que se han utilizado en otras estructuras enlazadas como **LinkedList** y **DLinkedList**.

Ambas clases son muy similares, las dos deben utilizar como atributos únicamente un puntero **current** que señala al nodo anterior al que se encuentra en el frente de la lista, y un atributo **size** para guardar la cantidad de elementos en la lista.

Parte 3

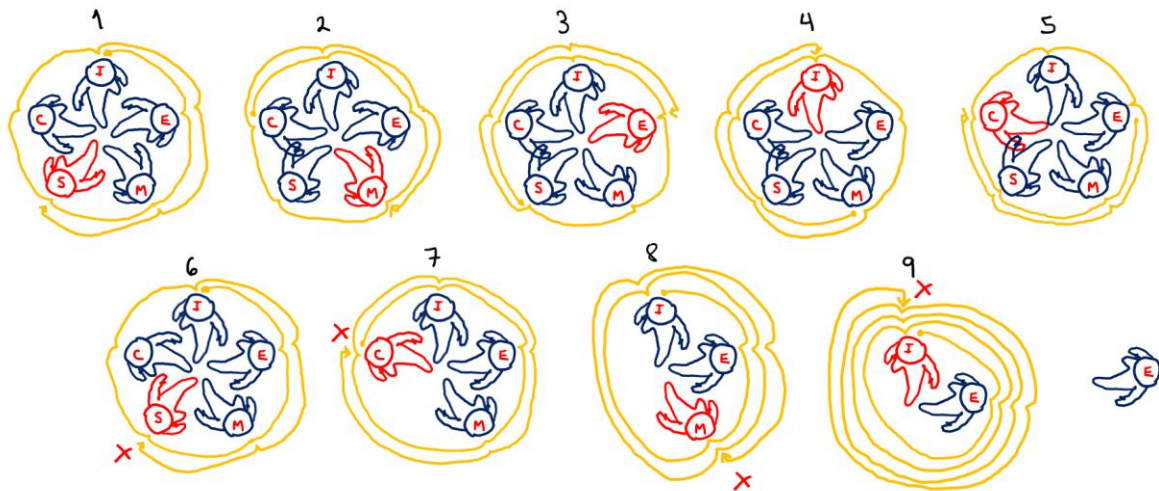
Esta parte consiste en implementar un **juego para escoger** que utilice listas circulares simples y dobles. Los juegos para escoger son fórmulas para **seleccionar una persona** que haga un determinado papel en un juego o realice alguna actividad. Estos juegos implementan una versión del **Problema de Flavio Josefo**, el cual consiste en elegir una persona de un grupo de personas dispuestas en un círculo a través de eliminaciones sucesivas, eligiéndolas a través de saltos de tamaño fijo.

El juego a implementar es el de **"Zapatito Cochinito"**, en el cual todas las personas ubican su pie derecho en un círculo. Se inicia de una persona cualquiera y se cuenta utilizando la siguiente rima:

"Zapa-tito-cochi-nito-cambia-de-pie-ci-to"

La rima se divide en 9 partes que representan 8 saltos a partir de la persona donde se inicia. A la persona que le caiga la sílaba final ("to") debe cambiar su pie derecho por el izquierdo, en caso de que no lo haya cambiado, o salir del círculo si ya lo había cambiado. Esto se repite múltiples veces hasta que sólo quede una persona en el círculo sin ser eliminada.

A modo de ejemplo, supongamos que cinco personas van a decidir quién va a iniciar una partida utilizando este método: Isabel, Eugenia, Mario, Sandra y Camilo. Todos inician con su pie derecho.



1. El conteo inicia en Isabel (se inicia por cualquier miembro) y termina en Sandra, por lo que Sandra cambia a su pie izquierdo.
2. El segundo conteo inicia en Carlos (porque está después de Sandra) y termina en Mario, por lo que Mario cambia de pie.
3. El tercero inicia en Sandra y termina en Eugenia, quien cambia de pie.
4. El cuarto inicia en Mario y termina en Isabel, quien cambia de pie.
5. El quinto inicia en Eugenia y termina en Carlos, quien cambia de pie.
6. El sexto inicia en Isabel y termina en Sandra, quien ya había cambiado de pie y por lo tanto sale del círculo. Esto cambia la cantidad de personas y por lo tanto el patrón de eliminación deja de ser tan claro.
7. El séptimo inicia en Carlos y termina en él mismo, que también sale del círculo.
8. El octavo inicia en Isabel y termina en Mario, quien sale del círculo.
9. El último conteo inicia en Isabel y termina en Isabel, quien sale del círculo y deja como elegida a Eugenia.

Escriba un programa que realice lo siguiente usando listas circulares:

1. Preguntar al usuario la cantidad de personas. Los nombres pueden ser asignados automáticamente según alguna lista predeterminada o puede solicitarlos al usuario.
2. Preguntar al usuario cuál tipo de lista desea utilizar: CircleList o DCircleList
3. Preguntar al usuario cuál modo quiere utilizar:
 - a. Regular: tal y como se explicó anteriormente. Los conteos se realizan siempre hacia la izquierda (en sentido de las manecillas del reloj).
 - b. Dirigido: cada vez que se va a hacer un conteo, el usuario elige en qué dirección quiere hacerlo, si hacia la izquierda (en el sentido de las manecillas del reloj) o hacia la derecha (en sentido contrario a las manecillas del reloj). En ambos casos el conteo debe empezar en la persona que se encuentre a la par de la última que fue elegida.
4. Implementar el juego tal y como se describió anteriormente hasta que sólo quede una persona. Para cada conteo:
 - a. Si está usando el modo dirigido:
 - i. Preguntar la dirección del siguiente conteo (izquierda o derecha)
 - b. Mostrar todos los elementos en la lista y cuál pie están usando actualmente

- c. Mostrar el resultado actualizado después del conteo
 - d. Mostrar claramente cuando una persona es eliminada del círculo
5. Al finalizar mostrar el nombre de la persona elegida

Entrega: Sección de evaluaciones del TEC-Digital.

Formato: Archivo comprimido ZIP con el proyecto completo de Visual Studio. Corrobore que se incluyen todos los archivos necesarios para compilar y ejecutar el proyecto. Recuerde limpiar los archivos de compilación del proyecto antes de comprimir y no incluir la carpeta oculta “.vs” dentro del ZIP.